Decidr

An AI Tutor for Theory of Computation

Kien Nguyen
Library and Learning Services
Sheridan College

July, 2026

Abstract

INFO 47546 - Theory of Computation is considered one of the most challenging courses at Sheridan College due to the abstract nature of its concepts and the emphasis on rigourous mathematical proofs.

Despite efforts to promote existing tutoring services, student engagement remains limited, with only one student seeking tutoring support during the Spring 2026 academic term. We identify several potential barriers such as scheduling conflicts, uncertainty regarding available resources, and hesitation in seeking academic assistance.

To address these challenges, we developed **Decidr**, an AI tutor chatbot designed to provide accessible, on-demand academic support. By leveraging **Retrieval-Augmented Generation (RAG)** techniques with course-specific materials, Decidr provides students with contextualized explanations and guidance grounded in INFO 47546 course content. The system aims to complement existing tutoring resources by lowering barriers to academic support and enabling students to receive assistance at their own convenience.

Contents

1 Introduction **2**

2 Background **3**
2.1 Large Language Models . . . 3
2.2 Retrieval-Augmented Generation . . . 3
2.3 Information Retrieval . . . 4
2.3.1 Sparse Retrieval . . . 4
2.3.2 Dense Retrieval . . . 5
2.3.3 Hybrid Retrieval . . . 5

3 System Architecture **6**
3.1 Overview . . . 6
3.2 Offline Indexing Pipeline . . . 7
3.2.1 Document Extraction . . . 7
3.2.2 Document Chunking . . . 7
3.2.3 Dense Index Construction . . . 7
3.2.4 Sparse Index Construction . . . 8
3.3 Online Inference Pipeline . . . 8
3.3.1 Query Preprocessing . . . 8
3.3.2 Hybrid Retrieval . . . 8
3.3.3 Prompt Construction . . . 9
3.3.4 Response Generation . . . 9

4 Iterative Development **10**
4.1 Local LLM Inference . . . 10
4.2 Fine-tuned LLM . . . 10
4.3 Adoption of RAG and Cloud Inference . . . 11
4.4 Lessons Learned . . . 12

5 Conclusion and Future Work **13**
5.1 Conclusion . . . 13
5.2 Future Work . . . 13

Chapter 1

Introduction

Theory of Computation is a foundational subject in computer science that introduces students to abstract computational models, including finite automata, formal languages, and computability theory. Unlike many programming-oriented courses, students are required to reason about mathematical structures (sets, functions, finite automata, etc.) and construct formal proofs to demonstrate correctness. For many students, this course represents a significant transition from intuitive problem-solving approaches toward rigorous mathematical thinking.

At Sheridan College, tutoring services are available to students at no additional cost. Tutors with sufficient background were made available for students enrolled in the course. However, traditional tutoring models often require students and tutors to coordinate schedules, which may create barriers for students with competing academic or personal commitments. Additionally, some students may hesitate to seek help due to uncertainty about available resources, or concerns about asking questions they perceive as basic.

Recent advancements in large language models (LLMs) have created new opportunities for developing intelligent educational tools capable of providing immediate and personalized assistance. However, general-purpose AI systems such as ChatGPT, Gemini and Claude may provide inaccurate or inconsistent responses when answering course-specific questions. Further, due to the limitations in context window size, uploading all of the course notes to these systems proved to be inefficient and not helpful. Therefore, an effective educational assistant should be grounded in reliable course materials while maintaining the flexibility of conversational interaction.

In response to these challenges, we developed **Decidr**, an AI-powered tutor chatbot for *INFO 47546 - Theory of Computation*. Decidr utilizes **Retrieval-Augmented Generation (RAG)** techniques to retrieve relevant information from course materials and incorporates this information into response generation, ensuring that explanations remain aligned with the curriculum.

Chapter 2

Background

2.1 Large Language Models

Large Language Models (LLMs) are a class of machine learning models designed to understand and generate human language [1]. These models are usually based on the *Transformer architecture* [2], and are trained on large-scale text datasets (for example, the Internet) to learn patterns, relationships, and representations within the language.

Thanks to their ability to generate coherent and contextually relevant text, LLMs have shown strong performance across a wide range of *Natural Language Processing (NLP)* tasks, including question answering, summarization, and conversational assistance.

The conversational capabilities of LLMs make them promising tools for educational applications. An LLM-based system can offer immediate responses, adapt explanations based on student questions, and support interactive learning experience.

However, general-purpose LLMs have several limitations when applied to domain-specific context. Since the knowledge of an LLM is derived from its training data, general-purpose models may not contain the specialized information required for a particular course. Furthermore, updating an LLM's internal knowledge through additional training can be computationally expensive and impractical for frequently changing information. Additionally, LLMs can generate plausible but incorrect reponses when it doesn't have an answer (a phenomenon referred to as *hallucination*). These limitations make it challenging to apply general-purpose LLMs directly to give course-specific information answers.

2.2 Retrieval-Augmented Generation

To address the limitations of LLMs outlined in the previous section, *Retrieval-Augmented Generation (RAG)* approaches combine LLMs with external knowledge retrieval mechanisms, allowing models to access relevant information during response generation.

A typical RAG pipeline consists of two main stages: *retrieval* and *generation*. During the *retrieval* stage, a user's query is used to identify relevant documents from an external knowledge base. These retrieved documents are then incorporated into the input prompt provided to the LLM. During the *generation* stage, the LLM uses both the

user's original query and the retrieved context to generate a response that is more accurate and grounded in the provided information.

By grounding responses in retrieved information, RAG reduces the likelihood of hallucination and allows LLM-based systems to incorporate domain-specific knowledge without requiring expensive model retraining. This makes RAG well-suited for applications such as educational assistants, where generated responses must remain consistent with specialized and potentially evolving course materials.

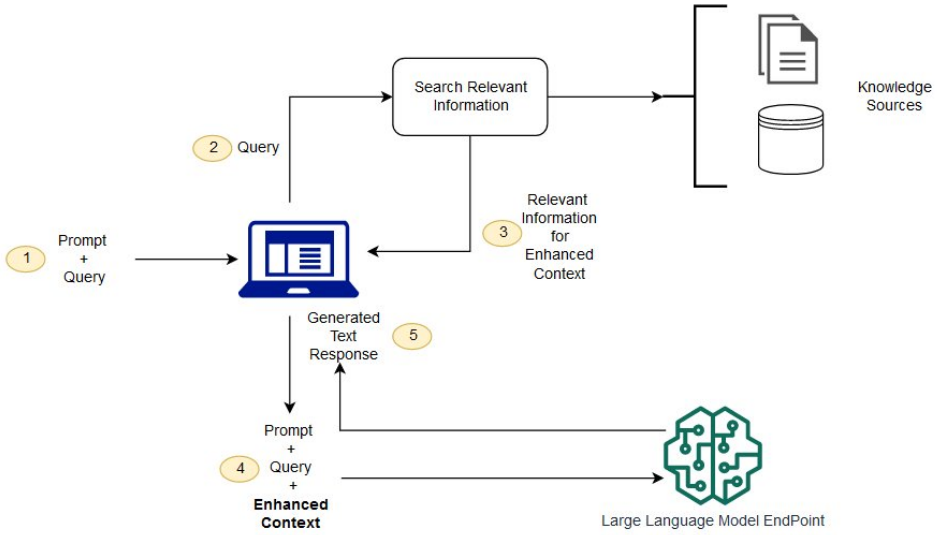


Figure 2.1: Architecture flow of Retrieval-Augmented Generation. Reprinted from [4].

2.3 Information Retrieval

Information Retrieval (IR) is a field in computer science concerned with finding documents or information that are relevant to a user's query. In RAG systems, the quality of retrieved documents directly influences the quality of the generated response. Modern retrieval systems generally employ either *sparse* retrieval, *dense* retrieval, or a combination of both (called *hybrid* retrieval) [5].

In general, an information retrieval system works as follows:

- Indexing: Process and organize documents into a searchable index.
- Retrieval: Given a user's query, rank indexed documents according to their estimated relevance and return the most relevant results.

The primary difference between *sparse* and *dense* retrieval lies in how documents and queries are represented during the indexing and retrieval stages.

2.3.1 Sparse Retrieval

Sparse retrieval methods represent documents as *keyword-based representations*. Rather than capturing the semantic meaning of text, sparse retrieval algorithms rank documents

based on the occurrence and importance of query terms. Consequently, these methods are particularly effective when a user's query closely matches the terminology contained within the indexed documents.

For example, if a user query contains the term "deterministic finite automaton", then sparse retrieval algorithms would strongly prioritize documents containing those exact words.

2.3.2 Dense Retrieval

Dense retrieval algorithms represent documents as *vectors* (called *embeddings*) that capture the documents' semantic meaning, rather than the exact words contained in them. This allows the retrieval system to identify relevant documents, even when different terminology is used.

For example, a query such as "machines that accept strings" may retrieve material discussing finite automata, even if those exact words do not appear.

2.3.3 Hybrid Retrieval

Sparse and dense retrieval exhibit complementary strengths. Sparse retrieval excels at exact terminology matching, while dense retrieval captures semantic similarity between related concepts. Consequently, many modern RAG systems employ hybrid retrieval strategies that combine both methods to improve retrieval accuracy.

Chapter 3

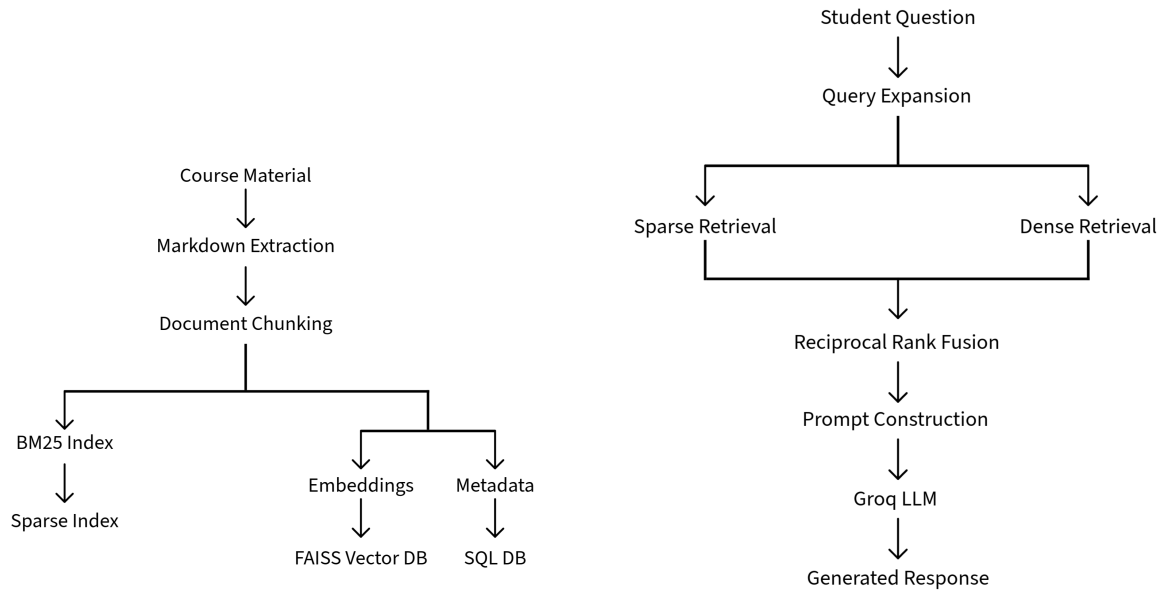
System Architecture

3.1 Overview

The overall architecture of Decidr consists of *two* primary pipelines:

- an offline *indexing* pipeline, and
- an online *inference* pipeline

The offline *indexing* pipeline processes course materials into a searchable knowledge base prior to deployment, while the online *inference* pipeline retrieves relevant course content in response to student queries and generates context-aware responses using an LLM.



(a) Offline indexing pipeline.

(b) Online inference pipeline.

Figure 3.1: Overview of Decidr's architecture. The offline pipeline constructs the searchable knowledge base, while the online pipeline retrieves relevant course materials and generates responses to student queries.

3.2 Offline Indexing Pipeline

Prior to deployment, these course materials are processed through an offline indexing pipeline to construct a searchable knowledge base. As illustrated in Figure 3.1a, the pipeline consists of document extraction, document chunking, and the construction of both dense and sparse retrieval indices.

3.2.1 Document Extraction

Course materials for the course INFO 47546 consists of 12 PDF files, corresponding to the 12 weeks of content for the semester.

Although PDFs are well suited for document presentation, they do not explicitly encode the logical structure of the content, making reliable extraction of headings, paragraphs, mathematical expressions, and other elements challenging.

To facilitate downstream processing, the lecture notes were converted to Markdown format using `PyMuPDF4LLM`, an open-source Python library designed to extract text from PDF documents while preserving their logical structure for large language model (LLM) applications. This conversion preserves section headings and document hierarchy while producing a machine-readable format that simplifies subsequent chunking and indexing.

3.2.2 Document Chunking

After extraction, the Markdown documents are divided into smaller chunks before indexing. Splitting documents into smaller segments allows the retrieval system to return only the portions of the course notes that are most relevant to a student's question, rather than the entire lecture notes. This finer retrieval granularity reduces the amount of irrelevant context supplied to the language model, resulting in more focused and contextually relevant responses.

Decidr uses the `MarkdownHeaderTextSplitter` provided by LangChain, which preserves the document hierarchy by splitting content according to Markdown headings. As a result, each chunk remains associated with its corresponding lecture section and subsection, providing additional context during retrieval.

3.2.3 Dense Index Construction

Each document chunk is converted into a dense vector representation using the open-source `all-MiniLM-L6-v2` embedding model. These embeddings capture the semantic meaning of the text, allowing similar concepts to be retrieved even when different wording is used.

The resulting embedding is stored is stored in a FAISS vector index [7] to support efficient similarity search. Alongside the vector index, a lightweight SQLite database stores the original document content and associated metadata (e.g., lecture week, section, and subsection). During retrieval, FAISS returns the identifies of the most relevant vectors, while the SQLite database is used to recover the corresponding document chunks supplied to the language model.

3.2.4 Sparse Index Construction

In addition to dense retrieval, Decidr constructs a sparse retrieval index using the Okapi BM25 algorithm [6]. Each document chunk is tokenized and added to a BM25 corpus, enabling efficient keyword-based retrieval. Sparse retrieval complements dense retrieval by emphasizing exact terminology frequently used throughout Theory of Computation, such as deterministic finite automaton or Pumping Lemma.

The offline indexing pipeline is executed only when course materials are updated. Once the dense and sparse indices have been constructed, they are reused by the online inference pipeline to answer student queries in real time.

3.3 Online Inference Pipeline

Once the offline indexing pipeline has constructed the searchable knowledge base, Decidr is capable of answering student queries in real time. As illustrated in Figure 3.1b, the online inference pipeline consists of query preprocessing, hybrid retrieval, prompt construction, and response generation.

3.3.1 Query Preprocessing

When a student submits a question, the query first undergoes a preprocessing stage before retrieval. During this stage, common abbreviations and acronyms used throughout the Theory of Computation course are expanded using a predefined lookup table. For example, "DFA" is expanded to "deterministic finite automaton", while "TM" is expanded to "Turing machine".

This preprocessing step improves retrieval performance by ensuring that both dense and sparse retrieval algorithms operate on queries containing complete terminology, reducing ambiguity and improving the likelihood of retrieving relevant course materials.

3.3.2 Hybrid Retrieval

Following preprocessing, the expanded query is submitted to a hybrid retrieval pipeline consisting of both dense and sparse retrieval. These retrieval strategies operate independently before their results are combined.

For dense retrieval, the expanded query is converted into a dense vector representation using the `all-MiniLM-L6-v2` embedding model. The resulting embedding is compared against the FAISS vector index using inner-product similarity, returning the top-k document chunks that are semantically similar to the student's query. By default, we set $k = 5$.

Simultaneously, the expanded query is tokenized and evaluated using the Okapi BM25 algorithm. Unlike dense retrieval, BM25 emphasizes exact keyword matching and is particularly effective when students use terminology that closely matches the course notes.

Finally, the ranked results produced by both retrieval methods are merged using *Reciprocal Rank Fusion (RRF)* [8]. Since dense and sparse retrieval produce scores on

different scales, RRF combines the rankings rather than the raw similarity scores, producing a unified ranking that benefits from the complementary strengths of both retrieval strategies.

3.3.3 Prompt Construction

Following reciprocal rank fusion, the identifiers of the highest-ranked document chunks are used to retrieve the corresponding document content and metadata from the SQLite database. These reconstructed document chunks are then incorporated into the prompt provided to the language model.

The system prompt instructs the language model to answer questions using only the retrieved course materials whenever possible, maintain consistency with the terminology used throughout the course, and explicitly acknowledge when the retrieved context does not contain sufficient information to answer the question. By grounding the language model in retrieved course content, the system encourages responses that remain aligned with the curriculum while reducing unsupported or fabricated answers.

3.3.4 Response Generation

The completed prompt is submitted to a hosted large language model (`llama-3.3-70b-versatile`) through the Groq inference API. The language model synthesizes the retrieved course materials with the student's question to produce a coherent, contextually relevant response. The generated response is then returned through Decidr's conversational interface, allowing students to receive immediate, course-specific assistance.

Chapter 4

Iterative Development

4.1 Local LLM Inference

During the initial stages of development, Decidr was designed to perform inference entirely on a local machine using a self-hosted large language model. Running the model locally eliminated dependency on external APIs, reduced operational costs, and provided complete control over the inference pipeline.

The initial prototype uses the `Llama3.1:8B` model by Meta, hosted through Ollama.

While the model was capable of producing coherent responses, inference latency proved to be a significant limitation. On a MacBook Pro equipped with Apple's M2 chip, generating a single response required approximately 15 seconds, resulting in an unsatisfactory user experience for an interactive tutoring application. Initially, we hypothesized that the limited memory available on the local machine was the primary performance bottleneck. To investigate this, the same model was deployed on a server equipped with substantially greater memory capacity. Surprisingly, inference time increased significantly, requiring approximately 30 seconds to generate a response. These experiments suggested that increasing available memory alone was insufficient to improve performance, and that inference throughput was instead constrained by computational resources available during model execution.

Since low response latency is critical for maintaining a conversational user experience, these observations motivated the exploration of alternative deployment strategies. In parallel, we also investigated whether the course-specific performance of the locally hosted language model could be improved through *parameter-efficient fine-tuning (PEFT)*.

4.2 Fine-tuned LLM

In simple terms, *parameter-efficient fine-tuning (PEFT)* is a family of techniques that adapt pre-trained language models to a specific task by updating only a small subset of the model's parameters while keeping the remaining parameters fixed. Compared with *full fine-tuning* (where we update all of the model's weights), this approach substantially reduces the computational resources required for training [9]. If successful, this approach could reduce the reliance on an external retrieval pipeline by incorporating course-specific

knowledge directly into the language model. This would simplify deployment by requiring only the fine-tuned model to be hosted during inference.

We applied PEFT using the *LoRA (Low-Rank Adaptation)* framework [10] to the `Llama3.1:8B` model in an effort to improve its performance on Theory of Computation questions while retaining the original model architecture. The model was fine-tuned on the *TuringQ* dataset [11] using the Python library `Unsloth`.

The fine-tuned model demonstrated improved familiarity with Theory of Computation terminology and question formats present in the training dataset. However, its overall performance remained inadequate for a conversational tutoring system. In particular, the model frequently struggled to synthesize information distributed across multiple retrieved document chunks, occasionally producing responses that confused unrelated concepts or lacked logical coherence.

Furthermore, the *TuringQ* dataset consists primarily of question-and-answer pairs collected from a variety of sources rather than Sheridan College's course materials. Consequently, although the model became more familiar with general Theory of Computation concepts, its responses often employed notation and terminology that differed from those used throughout the course. More importantly, the fine-tuned model continued to struggle with open-ended conceptual questions requiring multi-step reasoning, or the integration of information from multiple topics (e.g., "What is the difference between a DFA and an NFA?"). These observations suggested that parameter-efficient fine-tuning alone was insufficient for producing reliable, course-specific tutoring responses.

4.3 Adoption of RAG and Cloud Inference

The experiments with local inference and parameter-efficient fine-tuning revealed two important limitations. First, adapting a language model through fine-tuning alone was insufficient to ensure that responses remained consistent with Sheridan College's course materials. Second, locally hosted models introduced significant latency, limiting their suitability for an interactive tutoring application.

Based on these observations, we shifted our focus from modifying the language model itself toward improving the information provided to the model during response generation. Rather than attempting to encode course-specific knowledge directly into model parameters, the final version of Decidr adopted a Retrieval-Augmented Generation (RAG) architecture, where relevant course materials are dynamically retrieved and provided as context to the language model.

Groq is a cloud inference provider specializing in high-performance execution of large language models through its custom *Language Processing Unit (LPU)* architecture [12]. Unlike general-purpose GPUs, LPUs are designed specifically to accelerate the computational patterns involved in transformer inference, enabling high-throughput token generation and reduced response latency.

Since latency was identified as a major limitation of local deployment, Groq was se-

lected as the inference provider for the final version of Decidr. By leveraging Groq's optimized inference infrastructure, Decidr was able to utilize a larger language model while maintaining the responsiveness required for an interactive tutoring application.

4.4 Lessons Learned

The iterative development process of Decidr highlighted several important considerations when developing LLM-based educational applications. Initially, we focused on improving the language model itself through local deployment and parameter-efficient fine-tuning. However, these experiments demonstrated that model adaptation alone was insufficient for producing reliable, course-specific responses.

One key observation was that retrieval quality plays a critical role in domain-specific applications. While fine-tuning improved the model's familiarity with certain concepts and terminology, it did not provide a reliable mechanism for incorporating updated or institution-specific course materials. In contrast, Retrieval-Augmented Generation allowed Decidr to ground responses in authoritative course content without requiring repeated model training whenever course materials changed.

Additionally, the experiments highlighted the importance of balancing model capability with deployment constraints. Larger language models generally provide stronger reasoning capabilities, but their computational requirements make local deployment challenging. By combining a capable hosted language model with an efficient retrieval pipeline, Decidr was able to achieve a balance between response quality, course alignment, and inference latency.

The following table summarizes the approaches evaluated during the development of this project:

Approach	**Model**	**Strengths**	**Limitations**
Local inference	RAG + Llama 3.1 8B	No API dependency, easy experimentation	Slow inference, limited reasoning
LoRA fine-tuning	Llama 3.1 8B + LoRA	Better terminology familiarity	Poor generalization, notation mismatch
Final approach	RAG + llama-3.3-70b	Course grounding, strong reasoning, low latency	Requires external inference API

Table 4.1: Comparison of Modeling Approaches

Chapter 5

Conclusion and Future Work

5.1 Conclusion

This project presented **Decidr**, an AI-powered tutoring chatbot designed to provide accessible academic support for *INFO 47546 - Theory of Computation* at Sheridan College. The system addresses several challenges associated with traditional tutoring models, including scheduling constraints and uncertainty around seeking academic assistance.

Decidr combines Retrieval-Augmented Generation (RAG), hybrid information retrieval, and a large language model to provide students with course-specific explanations grounded in official lecture materials. Through the use of dense and sparse retrieval techniques, relevant course content can be identified and incorporated into the language model's context, improving the reliability and consistency of generated responses.

5.2 Future Work

Although Decidr demonstrates the potential of RAG-based educational assistants, several improvements could further enhance its effectiveness, reliability, and maintainability.

Currently, the retrieval corpus contains only lecture materials. Future work could expand the knowledge base to include additional course resources, such as exercises, assignments, and worked examples. Incorporating these materials would allow Decidr to provide more comprehensive assistance, particularly for students seeking help with problem-solving and practice questions.

The current query preprocessing stage relies on a manually constructed acronym lookup table for expanding common Theory of Computation terminology. A potential improvement would be to replace this static approach with LLM-based query expansion, where multiple related queries are generated dynamically. This could improve retrieval performance for ambiguous or complex student questions.

Furthermore, the retrieval pipeline could be enhanced through the addition of a re-ranking stage. While hybrid retrieval combines the complementary strengths of dense and sparse retrieval, a learned re-ranking model, such as a *cross-encode*r, could improve precision by evaluating the relevance of retrieved document chunks before they are provided to the

language model.

From a user experience perspective, future versions of Decidr could implement persistent conversation history using a database rather than temporary session storage. This would allow students to resume previous tutoring sessions and enable more coherent multi-session interactions. Finally, evaluation tooling could be developed to monitor system performance over time, including retrieval metrics such as recall and generation metrics such as answer faithfulness.

References

[1] Struker, C. "What are large language models (LLMs)?". *IBM*, https://www.ibm.com/think/topics/large-language-models

[2] Vaswani, A., Shazeer, N., Parmar, N., Uszkoreit, J., Jones, L., Gomez, A. N., Kaiser, L., Polosukhin, I. "Attention Is All You Need". *arXiv preprint*, 2017, arXiv:1706.03762.

[3] Amazon Web Services (AWS). "What is RAG (Retrieval-Augmented Generation)?". *Amazon Web Services* https://aws.amazon.com/what-is/retrieval-augmented-generation/#how-does-retrieval-augmented-generation-work--1xobboj

[4] Amazon Web Services (AWS). "Visualization of a RAG system". [Online Image]. *Amazon Web Services*, 2026. https://aws.amazon.com/what-is/retrieval-augmented-generation/#how-does-retrieval-augmented-generation-work--1xobboj

[5] Murel, J., Syed, M. "What is information retrieval?". *IBM*, https://www.ibm.com/think/topics/information-retrieval

[6] Park, E. "Understanding Okapi BM25 - Document Ranking algorithm". *Medium*, 2025. https://medium.com/@readwith_emma/understanding-okapi-bm25-document-ranking-algorithm-70d81adab001

[7] Douze, M., Guzhva, A., Deng, C., Johnson, J., Szilvasy, G., Mazaré P-E., Lomeli, M., Hosseini, L., Jégou, H. "The Faiss library". *arXiv preprint*, 2024, arXiv:2401.08281.

[8] Agarwal, S. "Search: Reciprocal Rank Fusion - Your Search Engine's Secret Weapon". *Medium*, 2025. https://shivamagarwal7.medium.com/search-reciprocal-rank-fusion-9735dcd1906d

[9] Belcic, I., Stryker, C. "What is parameter-efficient fine-tuning (PEFT)?". *IBM*, 2025. https://www.ibm.com/think/topics/parameter-efficient-fine-tuning

[10] Hu, E. J., Shen, Y., Wallis, P., Allen-Zhu, Z. Li, Y., Wang, S., Wang, L., Chen, W. "LoRA: Low-Rank Adaptation of Large Language Models". *arXiv preprint*, 2021, arXiv:2106.09685.

[11] Zahraei, P., Asgari, E. "TuringQ: Benchmarking AI Comprehension in Theory of Computation". *arXiv prepreint*, 2024, arXiv:2410.06547

[12] Groq. “What is a Language Processing Unit?”. *Groq*, 2024, https://cdn.sanity.io/files/chol0sk5/production/4e3e6966d98da9ef4bfd9834dbfc00921da58252.pdf